HTML — COMPLETE FROM ZERO

1. What exactly is HTML?

HTML = HyperText Markup Language

HTML is used to define the structure and content of a web page.

For example, if you want a webpage containing:

- a heading
- some text
- an image
- a link
- a table
- a form

HTML tells the browser what those things are and where they belong.

HTML itself does not primarily control how things look.

Think of a webpage like this:

```
HTML        → Structure
CSS         → Appearance
JavaScript  → Behaviour
```

For example:

```html
<h1>Student Registration</h1>
<p>Enter your details below.</p>
```

The browser understands:

> `<h1>` = *heading*

> `<p>` = *paragraph*

2. How HTML actually works

You create a file:

```
index.html
```

Inside it:

```html
<h1>Hello World</h1>
```

Then open the file in a browser.

The browser reads the HTML and renders:

Hello World

You don't need a server just to run basic HTML.

You can literally:

1. Open Notepad / VS Code
2. Write HTML
3. Save as `index.html`
4. Double-click it
5. It opens in Chrome/Edge

3. HTML Tags

A **tag** tells the browser what an element represents.

Example:

```html
<p>Hello</p>
```

Here:

```
<p>     → opening tag
Hello   → content
</p>    → closing tag
```

Together they form an HTML element.

```
<p>Hello</p>
└──────┘
   element
```

Another example:

```html
<h1>Welcome</h1>
```

4. Opening and Closing Tags

Most HTML elements have:

```html
<tag>content</tag>
```

Examples:

```html
<h1>Heading</h1>
<p>Paragraph</p>
<b>Bold</b>
<i>Italic</i>
```

But some HTML elements don't need closing tags.

These are called **void elements**.

Examples:

```html
<br>
<hr>
<img>
<input>
<meta>
<link>
```

Don't write:

```html
<br></br>
```

Normally just:

```html
<br>
```

5. Basic HTML Document Structure

This is the most important code you should memorize first.

```html
<!DOCTYPE html>

<html>
<head>
    <title>My Web Page</title>
</head>

<body>

    <h1>Hello World</h1>
    <p>This is my first webpage.</p>

</body>
</html>
```

Let's understand every line.

`<!DOCTYPE html>`

```html
<!DOCTYPE html>
```

Tells the browser:

> *This document uses modern HTML5.*

It is a declaration, not a normal HTML element.

`<html>`

```html
<html>
</html>
```

This is the root element.

Everything in the webpage goes inside `<html>`.

`<head>`

```html
<head>
</head>
```

Contains information about the webpage, rather than the visible page content.

For example:

```html
<title>
<meta>
<link>
<style>
```

`<title>`

```html
<title>My Web Page</title>
```

This appears in the browser tab.

It does not appear as the main heading inside the webpage.

`<body>`

```html
<body>
</body>
```

Contains the actual visible webpage content.

For example:

```html
<body>
    <h1>Welcome</h1>
    <p>Hello everyone.</p>
</body>
```

6. Comments

Comments are ignored by the browser.

```html
<!-- This is a comment -->
```

Example:

```html
<h1>Hello</h1>

<!-- Student details -->

<p>My name is Ajay.</p>
```

Comments are useful for explaining code.

7. HTML Attributes

Attributes provide **additional information** about an element.

Syntax:

```html
<tag attribute="value">
```

Example:

```html
<p id="para1">Hello</p>
```

Here:

```
p      → tag
id     → attribute
para1  → value
```

Another:

```html
<img src="photo.jpg" alt="Student Photo">
```

Attributes:

```
src → image location
alt → alternative text
```

Another:

```html
<a href="https://example.com">Visit</a>
```

`href` is an attribute.

8. Headings

HTML provides six heading levels.

```html
<h1>Heading 1</h1>
<h2>Heading 2</h2>
<h3>Heading 3</h3>
<h4>Heading 4</h4>
<h5>Heading 5</h5>
<h6>Heading 6</h6>
```

`h1` is the highest-level heading.

```
h1
↓
h2
↓
h3
↓
h4
↓
h5
↓
h6
```

Example:

```html
<h1>Computer Science</h1>
<h2>Web Technologies</h2>
<h3>HTML</h3>
```

9. Paragraph

```html
<p>This is a paragraph.</p>
```

Example:

```html
<p>
HTML is used to create the structure of a webpage.
</p>
```

10. Line Break

```html
<br>
```

Example:

```html
<p>Hello<br>World</p>
```

Output:

```
Hello
World
```

11. Horizontal Line

```html
<hr>
```

Example:

```html
<h1>HTML</h1>

<hr>

<p>HTML is a markup language.</p>
```

It creates a horizontal thematic line.

12. Text Formatting Tags

These are common basic HTML tags.

Bold

```html
<b>Bold text</b>
```

Italic

```html
<i>Italic text</i>
```

Underline

```html
<u>Underlined text</u>
```

Strong importance

```html
<strong>Important text</strong>
```

Emphasis

```html
<em>Emphasized text</em>
```

Highlight

```html
<mark>Highlighted text</mark>
```

Small text

```html
<small>Small text</small>
```

Deleted text

```html
<del>Deleted text</del>
```

Inserted text

```html
<ins>Inserted text</ins>
```

Subscript

```html
H<sub>2</sub>0
```

Displays: H_2O

Superscript

```html
x<sup>2</sup>
```

Displays: x^2

13. Links — `<a>`

The anchor element creates hyperlinks.

Basic syntax:

```html
<a href="URL">Link Text</a>
```

Example:

```html
<a href="https://www.google.com">Open Google</a>
```

When clicked, the browser navigates to that address.

Opening link in a new tab

```html
<a href="https://www.google.com" target="_blank">
    Open Google
</a>
```

`target="_blank"` opens a new browsing context, commonly a new tab.

14. Images — `<img>`

Basic syntax:

```html
<img src="image.jpg" alt="Description">
```

Example:

```html
<img src="student.jpg" alt="Student Photo">
```

Important attributes:

```
src    → image location
alt    → alternative text
width  → width
height → height
```

Example:

```html
<img src="student.jpg" alt="Student Photo" width="300" height="200">
```

15. Lists

HTML has three important types of lists.

Ordered List

Uses numbers.

```html
<ol>
    <li>HTML</li>
    <li>CSS</li>
    <li>JavaScript</li>
</ol>
```

Output:

```
1. HTML
2. CSS
3. JavaScript
```

Unordered List

Uses bullets.

```html
<ul>
    <li>HTML</li>
    <li>CSS</li>
    <li>JavaScript</li>
</ul>
```

Description List

Uses terms and descriptions.

```html
<dl>
    <dt>HTML</dt>
    <dd>Used for webpage structure.</dd>

    <dt>CSS</dt>
    <dd>Used for webpage styling.</dd>
</dl>
```

Here:

```
dl → description list
dt → description term
dd → description/details
```

16. Tables

This is important for coding questions.

Basic table:

```HTML
<table>
    <tr>
        <th>Name</th>
        <th>Age</th>
    </tr>

    <tr>
        <td>Ajay</td>
        <td>20</td>
    </tr>

    <tr>
        <td>Rahul</td>
        <td>21</td>
    </tr>
</table>
```

Understand these three:

```PLAIN TEXT
<table> → table
<tr>    → table row
<th>    → table heading
<td>    → table data
```

Table with border

For basic HTML coding practice:

```HTML
<table border="1">
    <tr>
        <th>Name</th>
        <th>Mark</th>
    </tr>

    <tr>
        <td>Ajay</td>
        <td>95</td>
    </tr>
</table>
```

Later, CSS is the proper way to style borders, but this helps you understand the structure.

17. `rowspan`

Allows a cell to span multiple rows.

```HTML
<table border="1">
    <tr>
        <th>Name</th>
        <th>Subject</th>
    </tr>

    <tr>
        <td rowspan="2">Ajay</td>
        <td>Java</td>
    </tr>

    <tr>
        <td>HTML</td>
    </tr>
</table>
```

The `Ajay` cell occupies two rows.

18. `colspan`

Allows a cell to span multiple columns.

```HTML
<table border="1">
    <tr>
        <th colspan="2">Student Details</th>
    </tr>

    <tr>
        <td>Name</td>
        <td>Ajay</td>
    </tr>
</table>
```

19. HTML `div`

`div` is a general-purpose block container.

```HTML
<div>
    <h1>Student</h1>
    <p>Student details</p>
</div>
```

It is extremely common when building webpages.
You will understand its styling much better when we start CSS.

20. HTML `span`

Your syllabus specifically mentions Span.
`span` is an inline container.
Example:

```HTML
<p>
    My favorite subject is
    <span>Web Technology</span>
</p>
```

It allows you to target a specific part of text.
For example:

```HTML
<p>
    My name is <span>Ajay</span>.
</p>
```

Later with CSS:

```HTML
<span style="color:red;">Ajay</span>
```

Only the `Ajay` portion is affected.
Remember:

```PLAIN TEXT
div  → block-level container
span → inline container
```

21. Forms

🔥🔥 This is one of the most important parts of your HTML syllabus.
A form collects input from the user.
Basic structure:

```HTML
<form>

    ...

</form>
```

Example:

```HTML
<form>
    <label>Name:</label>
    <input type="text">

    <br><br>

    <label>Email:</label>
    <input type="email">

    <br><br>

    <input type="submit" value="Register">
</form>
```

22. `<label>`

A label describes an input.

```HTML
<label>Name:</label>
<input type="text">
```

Better practice:

```HTML
<label for="name">Name:</label>
<input type="text" id="name">
```

The connection is:

```PLAIN TEXT
label for="name"
        ↓
input id="name"
```

They match.

23. Text Input

```HTML
<input type="text">
```

Example:

```HTML
<label>Name:</label>
<input type="text">
```

24. Password Input

```HTML
<input type="password">
```

Example:

```HTML
<label>Password:</label>
<input type="password">
```

The characters are hidden while typing.

25. Submit Button

```HTML
<input type="submit" value="Submit">
```

or:

```HTML
<button type="submit">Submit</button>
```

26. Reset Button

```HTML
<input type="reset" value="Reset">
```

It resets form controls to their initial values.

27. Radio Buttons

Used when selecting **one** option from a group.

```HTML
<label>Gender:</label>

<input type="radio" name="gender" value="male">
<label>Male</label>

<input type="radio" name="gender" value="female">
<label>Female</label>
```

Why is `name` important?
Both options have:

```HTML
name="gender"
```

Therefore the browser treats them as one group.

You can select one.

28. Checkboxes

Used when multiple options can be selected.

```HTML
<input type="checkbox" name="skill" value="html">
<label>HTML</label>

<input type="checkbox" name="skill" value="css">
<label>CSS</label>

<input type="checkbox" name="skill" value="js">
<label>JavaScript</label>
```

Unlike radio buttons, multiple checkboxes can be selected.

29. `<textarea>`

Used for multiline text.

```HTML
<textarea rows="5" cols="30"></textarea>
```

Example:

```HTML
<label>Address:</label>

<br>

<textarea rows="5" cols="30"></textarea>
```

30. `<select>` and `<option>`

Used to create a selection/drop-down control.

```HTML
<label>Department:</label>

<select>
    <option>CSE</option>
    <option>ECE</option>
    <option>EEE</option>
    <option>MECH</option>
</select>
```

The user selects one option.

31. Selected Default Option

```HTML
<select>
    <option>CSE</option>
    <option selected>ECE</option>
    <option>EEE</option>
</select>
```

`ECE` appears selected initially.

32. HTML5 New Input Types

Your syllabus specifically requires this.

The basic pattern is:

```HTML
<input type="...">
```

Let's go through the important HTML5 input types.

Email

```HTML
<input type="email">
```

Browser can perform basic email-format checking when the form is submitted.

Number

```HTML
<input type="number">
```

You can specify limits:

```HTML
<input type="number" min="18" max="60">
```

Date

```HTML
<input type="date">
```

Browser provides a date picker in supporting browsers.

Time

```HTML
<input type="time">
```

Date and Time

Modern HTML uses:

```HTML
<input type="datetime-local">
```

Month

```HTML
<input type="month">
```

Week

```HTML
<input type="week">
```

Color

```HTML
<input type="color">
```

Range

```HTML
<input type="range" min="0" max="100">
```

Creates a slider.

URL

```HTML
<input type="url">
```

Used for URLs.

Search

```HTML
<input type="search">
```

Telephone

```HTML
<input type="tel">
```

File

```HTML
<input type="file">
```

Allows the user to choose a file.

33. Important Input Attributes

You should know these because coding questions frequently combine them with input types.

`name`

```HTML
<input type="text" name="username">
```

Gives the control a name.

`id`

```HTML
<input type="text" id="username">
```

Identifies the element.

`value`

```HTML
<input type="text" value="Ajay">
```

Provides an initial value.

`placeholder`

```HTML
<input type="text" placeholder="Enter your name">
```

Displays a hint.

`required`

```HTML
<input type="text" required>
```

The field must be filled before normal form submission can proceed.

`readonly`

```HTML
<input type="text" value="CSE" readonly>
```

User can see the value but normally cannot edit it.

`disabled`

```HTML
<input type="text" disabled>
```

The control is disabled.

`min`

```HTML
<input type="number" min="1">
```

`max`

```HTML
<input type="number" max="100">
```

`step`

```HTML
<input type="number" min="0" max="100" step="5">
```

34. `autocomplete`

Your syllabus specifically mentions:

> *Auto Complete Attributes*

Example:

```HTML
<input type="text" autocomplete="on">
```

Or:

```HTML
<form autocomplete="on">
```

You can also disable autocomplete:

```HTML
<form autocomplete="off">
```

The browser may use previously entered information to provide suggestions.

35. `<datalist>`

🔥 This is specifically in your syllabus.

`datalist` provides a list of suggested values for an input.

Example:

```HTML
<label for="city">City:</label>

<input list="cities" id="city">

<datalist id="cities">
    <option value="Chennai">
    <option value="Coimbatore">
    <option value="Madurai">
    <option value="Salem">
</datalist>
```

Here's the important relationship:

```
<input list="cities">
        ↓
<datalist id="cities">
```

The values match:

```
list="cities"
      =
id="cities"
```

When the user clicks/types in the input, the browser can show the suggestions.

36. `autocomplete` + `datalist` Together

This is a very good exam-style example.

```html
<form autocomplete="on">

    <label for="city">Choose City:</label>

    <input list="cities" id="city" name="city">

    <datalist id="cities">
        <option value="Chennai">
        <option value="Coimbatore">
        <option value="Madurai">
        <option value="Salem">
    </datalist>

    <input type="submit">

</form>
```

Understand that these are two different mechanisms:

```
datalist
   ↓
developer-provided suggestions

autocomplete
   ↓
browser-provided remembered suggestions
```

They can be used together.

37. Form `action` and `method`

Even though your current syllabus is primarily HTML forms, you should understand these because they are part of the actual form syntax.

```html
<form action="/submit" method="post">
```

↓

`action`

Specifies where form data should be sent.

`method`

Specifies how the data is submitted.

Common methods:

```
GET
POST
```

Example:

```html
<form action="/register" method="post">

    <label>Name:</label>
    <input type="text" name="name">

    <input type="submit">

</form>
```

You don't need a backend for understanding the HTML syntax.

38. `fieldset` and `legend`

Useful for organizing forms.

```html
<fieldset>

    <legend>Student Details</legend>

    <label>Name:</label>
    <input type="text">

    <br><br>

    <label>Age:</label>
    <input type="number">

</fieldset>
```

It visually groups related form controls.

39. Semantic HTML

HTML5 introduced semantic elements that describe the meaning of page sections.

Common ones:

```html
<header>
<nav>
<main>
<section>
<article>
<aside>
<footer>
```

Example:

```html
<header>
    <h1>My Website</h1>
</header>

<nav>
    <a href="#">Home</a>
    <a href="#">About</a>
</nav>

<main>
    <section>
        <h2>About Me</h2>
        <p>This is my webpage.</p>
    </section>
</main>

<footer>
    Copyright 2026
</footer>
```

These aren't the main focus of your listed syllabus, but they're useful HTML5 knowledge for coding.

40. HTML Entities

Sometimes you want to display special characters.

For example:

```
&lt;
&gt;
&amp;
 
&quot;
```

They represent:

```
&lt;    <
&gt;    >
&amp;   &
   space
&quot;  "
```

Example:

```html
<p>5 &lt; 10</p>
```

Displays:

```
5 < 10
```

41. Nesting HTML Elements

HTML elements can be placed inside other elements.

✅ Correct:

```html
<p>
    This is <b>important</b>.
</p>
```

Here:

```
p
└── b
```

Another:

```html
<div>
    <h1>Student</h1>
    <p>Hello</p>
</div>
```

This is called nesting.

Tags should be properly closed.

❌ Wrong:

```html
<p><b>Hello</p></b>
```

✅ Correct:

```html
<p><b>Hello</b></p>
```

42. A Complete HTML Coding Example

Now let's combine what you've learned.

This is the kind of program you should eventually be able to write in your exam without copying.

```html
<!DOCTYPE html>

<html>

<head>
    <title>Student Registration</title>
</head>

<body>

    <h1>Student Registration Form</h1>

    <hr>

    <form autocomplete="on">

        <label for="name">Name:</label>
        <input type="text" id="name" name="name"
               placeholder="Enter your name" required>

        <br><br>

        <label for="email">Email:</label>
        <input type="email" id="email" name="email" required>
```

```
        <br><br>

        <label>Gender:</label>
        <input type="radio" name="gender" value="male"> Male
        <input type="radio" name="gender" value="female"> Female

        <br><br>

        <label for="dept">Department:</label>
        <select id="dept" name="dept">
            <option>CSE</option>
            <option>ECE</option>
            <option>EEE</option>
        </select>

        <br><br>

        <input type="submit" value="Register">

    </form>

</body>

</html>
```

If you save this as:

```
student.html
```

and open it in Chrome/Edge, you'll get a working student registration webpage.

43. Now understand the whole program structurally

Don't memorize that entire program as one giant thing.
Understand its structure:

```
<!DOCTYPE html>
    ↓
  <html>
    ↓
┌──────────┐
│   head   │
│   title  │
└──────────┘
    ↓
┌──────────┐
│   body   │
│          │
│ heading  │
│ form     │
└──────────┘
  │ text input   │
  │ email input  │
  │ number input │
  │ radio        │
  │ checkbox     │
  │ datalist     │
  │ select       │
  │ date         │
  │ color        │
  │ textarea     │
  │ submit       │
  │ reset        │
  └──────────┘
    ↓
  </html>
```

If you understand this hierarchy, writing HTML in the exam becomes much easier.

44. The HTML Syntax Patterns You Should Know by Heart

These are the patterns I want you to be able to write without looking at notes.

Basic element

```
<tag>content</tag>
```

Attribute

```
<tag attribute="value">
```

Heading

```
<h1>Heading</h1>
```

Paragraph

```
<p>Paragraph</p>
```

Link

```
<a href="URL">Text</a>
```

Image

```
<img src="image.jpg" alt="Image">
```

Ordered list

```
<ol>
    <li>Item</li>
</ol>
```

Unordered list

```
<ul>
    <li>Item</li>
</ul>
```

Table

```
<table>
    <tr>
        <th>Heading</th>
    </tr>
    <tr>
        <td>Data</td>
    </tr>
</table>
```

Form

```
<form>
    ...
</form>
```

Text input

```
<input type="text">
```

Radio

```
<input type="radio" name="x">
```

Checkbox

```
<input type="checkbox" name="x">
```

Select

```
<select>
    <option>Option</option>
</select>
```

Textarea

```
<textarea></textarea>
```

Submit

```
<input type="submit" value="Submit">
```

Datalist

```
<input list="items">

<datalist id="items">
    <option value="A">
    <option value="B">
</datalist>
```

HTML5 input

```
<input type="email">
<input type="number">
<input type="date">
<input type="time">
<input type="color">
<input type="range">
<input type="url">
```

🧠 The most important thing for your coding exam

You don't need to memorize 100 unrelated tags.
You need to understand this:

```
HTML DOCUMENT
  │
  ├── HEAD
  │     └── TITLE
  │
  └── BODY
        │
        ├── TEXT
        ├── HEADINGS
        ├── LINKS
        ├── IMAGES
        ├── LISTS
        ├── TABLES
        ├── DIV / SPAN
        │
        └── FORM
              ├── INPUT
              ├── RADIO
              ├── CHECKBOX
              ├── SELECT
              ├── TEXTAREA
              ├── HTML5 INPUT TYPES
              ├── DATALIST
              └── SUBMIT / RESET
```

CSS — COMPLETE CODING GUIDE

1. What is CSS?

CSS = Cascading Style Sheets
HTML creates the structure:

```
<h1>Student Registration</h1>
<p>Enter your details.</p>
```

CSS controls how those elements look:

```css
h1 {
    color: blue;
    font-size: 30px;
}

p {
    color: grey;
}
```

So:

```
HTML
 ↓
What is on the page?

CSS
 ↓
How does it look and where is it placed?
```

2. Basic CSS Syntax

This is the most fundamental CSS syntax:

```css
selector {
    property: value;
}
```

Example:

```css
p {
    color: red;
    font-size: 20px;
}
```

Break it down:

```
p
↓
selector

color
↓
property

red
↓
value
```

The entire:

```css
color: red;
```

is a **declaration**.

Multiple declarations are written inside `{ }`.

```css
p {
    color: red;
    font-size: 20px;
    text-align: center;
}
```

3. CSS Comments

```css
/* This is a CSS comment */
```

Example:

```css
p {
    /* Changes text color */
    color: blue;
}
```

4. Selectors

Before you can properly understand CSS, you need to understand what element you're styling.

That is the job of a **selector**.

Element Selector

Selects all elements of a particular type.

```css
p {
    color: red;
}
```

Every `<p>` becomes red.

Multiple Element Selectors

```css
h1, h2, p {
    color: blue;
}
```

This applies the same style to:

```
h1
h2
p
```

5. Class Selector

HTML:

```html
<p class="important">Important message</p>
<p>Normal message</p>
```

CSS:

```css
.important {
    color: red;
}
```

The `.` means class.

Only elements having that class are selected.

You can use the same class on multiple elements:

```html
<p class="important">Important message</p>
<h2 class="important">Warning</h2>
```

6. ID Selector

HTML:

```html
<p id="special">Hello</p>
```

CSS:

```css
#special {
    color: blue;
}
```

The `#` means ID.

Think:

```
.abc → class
#abc → id
```

7. Universal Selector

```css
* {
    margin: 0;
}
```

`*` selects all elements.

8. Basic CSS Properties

Now we can start actually styling pages.

Color

```css
p {
    color: red;
}
```

You can use names:

```css
color: blue;
```

Hexadecimal:

```css
color: #ff0000;
```

RGB:

```css
color: rgb(255, 0, 0);
```

9. Background Color

```css
body {
    background-color: lightgray;
}
```

Or:

```css
div {
    background-color: blue;
}
```

10. Font Size

```css
p {
    font-size: 20px;
}
```

11. Font Family

```css
p {
    font-family: Arial;
}
```

Multiple fallback fonts:

```css
p {
    font-family: Arial, sans-serif;
}
```

12. Font Weight

```css
p {
    font-weight: bold;
}
```

Or:

```css
p {
    font-weight: 700;
}
```

13. Font Style

```css
p {
    font-style: italic;
}
```

14. Text Alignment

```css
h1 {
    text-align: center;
}
```

Other values:

```css
text-align: left;
text-align: right;
text-align: center;
text-align: justify;
```

15. Text Decoration

```css
p {
    text-decoration: underline;
}
```

Other examples:

```css
text-decoration: none;
text-decoration: line-through;
```

This is useful for removing the default underline from links:

```css
a {
    text-decoration: none;
}
```

16. Width and Height

```css
div {
    width: 300px;
    height: 200px;
}
```

17. Border

Basic:

```css
div {
    border: 2px solid black;
}
```

Breakdown:

```
2px    → thickness
solid  → style
black  → color
```

Other border styles:

```css
border: 2px dashed black;
border: 2px dotted black;
```

18. Border Radius

Makes corners rounded.

```css
div {
    border-radius: 10px;
}
```

Circle:

```css
img {
    border-radius: 50%;
}
```

19. CSS3

Your syllabus specifically says CSS3.

CSS3 introduced/expanded modern CSS capabilities such as:

- rounded corners
- shadows
- gradients
- transitions
- animations
- media queries
- more flexible layouts

For your specific syllabus, **media queries** are particularly important because they're explicitly listed.

You don't need to treat every modern CSS feature as an additional syllabus topic.

20. Introduction to Cascading Style

Now we reach an important concept.

Why is CSS called **Cascading**?

Because multiple CSS rules can target the same element.

Suppose:

```html
<p class="text">Hello</p>
```

And:

```css
p {
    color: blue;
}

.text {
    color: red;
}
```

Both rules apply to the same `<p>`.

CSS needs a mechanism for deciding which declaration wins.

This is where the cascade comes in.

21. Types of Stylesheets

Your syllabus specifically lists three.

There are three ways to apply CSS:

```
Inline
Embedded
External
```

22. Inline Styles

CSS is written directly inside the HTML element.

```html
<p style="color: red; font-size: 20px;">
    Hello World
</p>
```

Here:

```
style="..."
```

contains the CSS.

Another example:

```html
<h1 style="color: blue; text-align: center;">
    Web Technology
</h1>
```

Advantages

Useful for applying a style to one specific element.

Disadvantage

If you have 100 elements, writing styles individually becomes messy.

23. Embedded Style Sheets

CSS is written inside a `<style>` element, normally in `<head>`.

```html
<!DOCTYPE html>
<html>

<head>

    <style>
        h1 {
            color: blue;
        }

        p {
            font-size: 20px;
        }
    </style>

</head>

<body>

    <h1>Hello</h1>
    <p>Welcome to CSS.</p>

</body>
</html>
```

The CSS applies to the page containing that `<style>` block.

24. External Style Sheets

This is the most useful approach for larger websites.

Create:

```
index.html
style.css
```

style.css

```css
h1 {
    color: blue;
}

p {
    font-size: 20px;
}
```

index.html

```html
<!DOCTYPE html>
<html>

<head>
    <title>My Website</title>
    <link rel="stylesheet" href="style.css">
</head>

<body>

    <h1>Hello</h1>
    <p>Welcome.</p>

</body>

</html>
```

The important line is:

```html
<link rel="stylesheet" href="style.css">
```

25. Difference Between the Three Stylesheets

Type	CSS Location	Scope
Inline	Inside HTML element	One element
Embedded	`<style>` in HTML	One page
External	Separate `.css` file	Multiple pages

For an exam, you should be able to write the syntax for all three.

26. Issues in Styling

This part of your syllabus connects directly with cascading.

The main problem is:

> *What happens when multiple CSS rules affect the same element?*

Example:

```html
<p class="message" id="important">Hello</p>
```

CSS:

```css
p {
    color: blue;
}

.message {
    color: green;
}

#important {
    color: red;
}
```

The paragraph ultimately becomes **red** because the ID selector has greater specificity than the class and element selectors.

The basic specificity hierarchy you should understand is:

```
Inline style
     ↓
    ID
     ↓
   Class
     ↓
  Element
```

So:

```css
p {
    color: blue;
}

.message {
    color: green;
}

#important {
    color: red;
}
```

results in red.

27. `!important`

CSS also has:

```css
p {
    color: red !important;
}
```

`!important` gives that declaration very high priority in the cascade.

You should know what it means, but don't use it everywhere.

28. Positioning Elements

🔥 This is a major exam topic.

CSS positioning is controlled using:

```css
position:
```

Your syllabus specifically contains:

```
Static
Relative
Fixed
Absolute
```

29. Static Positioning

```css
div {
    position: static;
}
```

This is the default positioning.

The element stays in the normal document flow.

For example:

```html
<div class="box">Box</div>
```

```css
.box {
    position: static;
    top: 50px;
}
```

`top: 50px` doesn't move a normally static element.

30. Relative Positioning

```css
.box {
    position: relative;
    top: 50px;
    left: 30px;
}
```

The element moves **relative to its normal position**.

Important:

The original space occupied by the element is generally preserved in the document flow.

Imagine:

```
Original position
        ↓
    [ BOX ]

            ↓ move right/down

          [ BOX ]
```

31. Absolute Positioning

```css
.box {
    position: absolute;
    top: 50px;
    left: 100px;
}
```

An absolutely positioned element is removed from the normal document flow.

Its position is determined relative to an appropriate positioned ancestor; if none exists, its containing block is typically the initial containing block.

This is extremely important.

32. Absolute Positioning with a Parent

Consider:

```html
<div class="container">
    <div class="box">Box</div>
</div>
```

CSS:

```css
.container {
    position: relative;
    width: 500px;
    height: 300px;
    border: 2px solid black;
}

.box {
    position: absolute;
    top: 20px;
    left: 30px;
}
```

Here:

```
container
    ↓
position: relative

box
   ↓
position: absolute
```

The absolute child is positioned relative to the positioned container.

This parent-child relationship is very important for coding questions.

33. Fixed Positioning

```css
.box {
    position: fixed;
    top: 20px;
    right: 20px;
}
```

The element is positioned relative to the viewport and generally stays in place when the page scrolls.

For example, a fixed button:

```html
<button class="help">Help</button>
```

```css
.help {
    position: fixed;
    bottom: 20px;
    right: 20px;
}
```

The button stays near the bottom-right of the viewport while scrolling.

34. Positioning Comparison

Position	Normal Flow?	Typical Reference
`static`	Yes	Normal document flow
`relative`	Yes	Its normal position
`absolute`	No	Positioned containing block
`fixed`	No	Viewport

You should know this table extremely well.

35. `top`, `right`, `bottom`, `left`

These properties are used with positioned elements.

```css
.box {
    position: relative;
    top: 20px;
    left: 30px;
}
```

You can use:

```css
top
right
bottom
left
```

Example:

```css
.box {
    position: absolute;
    right: 20px;
    bottom: 20px;
}
```

36. Z-index

When elements overlap, which one appears on top?

That's where:

```css
z-index
```

comes in.

Example:

```html
<div class="box1">Box 1</div>
<div class="box2">Box 2</div>
```

```css
.box1 {
    position: absolute;
    width: 150px;
    height: 150px;
    background: red;
    z-index: 1;
}

.box2 {
    position: absolute;
    width: 150px;
    height: 150px;
    background: blue;
    top: 50px;
    left: 50px;
    z-index: 2;
}
```

Since:

```
box2 z-index = 2
box1 z-index = 1
```

Box 2 is painted above Box 1, assuming the elements participate in the same relevant stacking context.

37. Span

You already saw `<span>` in HTML.

Now understand why it matters in CSS.

HTML:

```html
<p>
    This is <span class="important">important</span>.
</p>
```

CSS:

```css
.important {
    color: red;
    font-weight: bold;
}
```

Only:

```
important
```

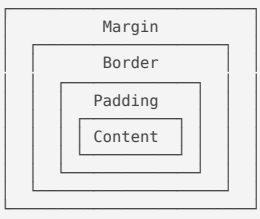
gets the styling.

`span` is useful when you need to style a small inline portion of content.

38. Box Model

🔥🔥 Extremely important

Every element can be understood using the CSS box model.



The four layers are:

```
Content
Padding
Border
Margin
```

39. Content

The actual content.

```css
.box {
    width: 200px;
    height: 100px;
}
```

The content area is where text/images/etc. are placed.

40. Padding

Space inside the element, between content and border.

```css
.box {
    padding: 20px;
}
```

Think:

```
Border
   ↓
Padding
   ↓
Content
```

41. Border

Surrounds the padding/content area.

```css
.box {
    border: 2px solid black;
}
```

42. Margin

Space outside the border.

```css
.box {
    margin: 20px;
}
```

Think:

```
Other element

   ← margin →

    border
    padding
    content
```

43. Different Padding Syntaxes

All sides:

```css
padding: 20px;
```

Top/bottom and left/right:

```css
padding: 10px 20px;
```

This means:

```
top/bottom = 10px
left/right = 20px
```

Four values:

```css
padding: 10px 20px 30px 40px;
```

Order:

```
top
right
bottom
left
```

Same pattern applies to margin:

```css
margin: 10px 20px 30px 40px;
```

44. Individual Box Model Properties

You can write:

```css
.box {
    margin-top: 10px;
    margin-right: 20px;
    margin-bottom: 30px;
    margin-left: 40px;
}
```

Likewise:

```css
.box {
    padding-top: 10px;
    padding-right: 20px;
    padding-bottom: 30px;
    padding-left: 40px;
}
```

45. `box-sizing`

Very useful when calculating element sizes.

```css
* {
    box-sizing: border-box;
}
```

With `border-box`, the declared width/height includes the content, padding, and border within that box.

Example:

```css
.box {
    width: 300px;
    padding: 20px;
    border: 5px solid black;
    box-sizing: border-box;
}
```

The total rendered width is constrained to 300px.

46. Text Flow

Text flow refers to how text is laid out around and within elements.

A classic example is floating an image:

```html
<img src="photo.jpg" class="photo">

<p>
    This is some text that flows around the image.
</p>
```

```css
.photo {
    float: left;
    margin-right: 20px;
}
```

The text can flow alongside the floated image.

You should understand this concept because "Box Model and Text Flow" is explicitly in your syllabus.

47. Media Types

CSS can target different media types.

For example:

```css
@media screen {
    body {
        background-color: white;
    }
}
```

Common media types include:

```
screen
print
```

For example:

```css
@media print {
    body {
        font-size: 12pt;
    }
}
```

This rule is intended for printed output.

48. Media Queries

🔥🔥 Important coding topic.

Media queries allow CSS to apply conditionally depending on characteristics such as viewport width.

Basic syntax:

```css
@media screen and (max-width: 600px) {

    body {
        background-color: lightgray;
    }

}
```

Meaning:

> *If the screen viewport is 600px wide or less, apply these rules.*

49. Example of Responsive Design

HTML:

```html
<div class="box">
    Responsive Box
</div>
```

CSS:

```css
.box {
    width: 500px;
    height: 200px;
    background-color: blue;
}

@media screen and (max-width: 600px) {

    .box {
        width: 100%;
        height: 150px;
    }

}
```

On a large screen:

```
width = 500px
```

On a screen ≤ 600px:

```
width = 100%
```

This is the basic idea of responsive design.

50. `min-width` Media Query

```css
@media screen and (min-width: 800px) {

    body {
        font-size: 20px;
    }

}
```

The styles apply when the viewport is at least 800px wide.

51. Combining Conditions

```css
@media screen and (min-width: 600px) and (max-width: 1000px) {

    .box {
        width: 70%;
    }

}
```

Both conditions must be satisfied.

52. Drop-Down Menus

Your final CSS syllabus topic is **Drop-Down Menus**.

There are different ways to implement one.

The simplest HTML-native dropdown is:

```html
<select>
    <option>Java</option>
    <option>Python</option>
    <option>JavaScript</option>
</select>
```

But a CSS navigation dropdown is more useful for understanding CSS positioning and hover behavior.

53. CSS Drop-Down Menu

HTML:

```html
<ul class="menu">

    <li>
        <a href="#">Home</a>
    </li>

    <li class="dropdown">

        <a href="#">Courses</a>

        <div class="dropdown-content">

            <a href="#">HTML</a>
            <a href="#">CSS</a>
            <a href="#">JavaScript</a>

        </div>

    </li>

</ul>
```

CSS:

```css
.menu {
    list-style-type: none;
    margin: 0;
    padding: 0;
}

.menu li {
    display: inline-block;
    position: relative;
}

.menu a {
    display: block;
    padding: 10px 20px;
    text-decoration: none;
}

.dropdown-content {
    display: none;
    position: absolute;
    top: 100%;
    left: 0;
    background-color: white;
    min-width: 150px;
}

.dropdown-content a {
    display: block;
}

.dropdown:hover .dropdown-content {
    display: block;
}
```

54. Understand How That Dropdown Actually Works

This is important for the exam.

Initially:

```css
.dropdown-content {
    display: none;
}
```

So the submenu is hidden.

Then:

```css
.dropdown:hover .dropdown-content {
    display: block;
}
```

When the user hovers over `.dropdown`:

```
.dropdown
    ↓ hover
.dropdown-content
    ↓
display: block
```

The submenu appears.

And:

```css
.dropdown {
    position: relative;
}
```

combined with:

```css
.dropdown-content {
    position: absolute;
}
```

makes the submenu position relative to the dropdown item.

So your dropdown menu combines several syllabus concepts:

- Positioning
- Absolute positioning
- CSS selectors
- Span/HTML elements
- Styling
- Hover
- Display
- Drop-down menu

55. One Complete CSS Exam-Style Program

Now let's combine your HTML + CSS knowledge.

index.html

```html
<!DOCTYPE html>

<html>

<head>

    <title>Student Page</title>

    <link rel="stylesheet" href="style.css">

</head>

<body>

    <h1>Student Registration</h1>

    <div class="container">

        <h2>Personal Details</h2>

        <form>

            <label>Name:</label>
            <input type="text">

            <label>Email:</label>
            <input type="email">

            <label>Age:</label>
            <input type="number">

            <label>Department:</label>

            <select>
                <option>CSE</option>
                <option>ECE</option>
                <option>EEE</option>
            </select>

            <button type="submit">Register</button>

        </form>

    </div>

</body>

</html>
```

style.css

```css
body {
    font-family: Arial, sans-serif;
    background-color: lightgray;
}

h1 {
    text-align: center;
    color: darkblue;
}

.container {
    width: 400px;
    margin: 30px auto;
    padding: 20px;
    background-color: white;
    border: 2px solid black;
    border-radius: 10px;
}

h2 {
    text-align: center;
}

label {
    display: block;
    margin-top: 10px;
}

input,
select {
    width: 100%;
    padding: 8px;
    margin-top: 5px;
    box-sizing: border-box;
}

button {
    margin-top: 20px;
    padding: 10px 20px;
}
```

This gives you an actual webpage with:

```
HTML
 ↓
Structure

CSS
 ↓
Colors
Fonts
Spacing
Border
Box model
Sizing
Alignment
```

🧠 The CSS mental model you need for the exam

Don't try to memorize CSS as hundreds of isolated properties.

Think of every CSS rule as:

```
WHO?
 ↓
selector

WHAT?
 ↓
property

HOW?
 ↓
value
```

Example:

```css
.student {
    color: blue;
}
```

means:

```
WHO?  → .student
WHAT? → color
HOW?  → blue
```

Then your major CSS concepts become:

```
CSS
├── Syntax
│
├── Selectors
│
├── Stylesheets
│     ├── Inline
│     ├── Embedded
│     └── External
│
├── Cascade / Styling conflicts
│
├── Positioning
│     ├── Static
│     ├── Relative
│     ├── Absolute
│     └── Fixed
│
├── Z-index
│
├── Span
│
├── Box Model
│     ├── Content
│     ├── Padding
│     ├── Border
│     └── Margin
│
├── Text Flow
│
├── Media
│     ├── Media Types
│     └── Media Queries
│
└── Drop-down Menus
```

JAVASCRIPT — COMPLETE CODING GUIDE

1. JavaScript: History and Versions

This is mainly theory, but you should know the basics.

What is JavaScript?

JavaScript is a high-level programming language mainly used to add programming behavior to web pages.

You already have:

```
HTML → Structure
CSS → Presentation
JavaScript → Behaviour / Programming
```

For example:

```javascript
let age = 20;

if (age >= 18) {
    console.log("Adult");
}
```

JavaScript can perform calculations, make decisions, repeat operations, manipulate data, and much more.

JavaScript vs Java

Don't confuse these.

```
JavaScript ≠ Java
```

They are different programming languages.

ECMAScript

ECMAScript is the standardized specification on which JavaScript is based.

You will commonly see:

```
ES5
ES6
ES2015
ES2016
...
```

A very important version is:

> *ES6 = ECMAScript 2015*

ES6 introduced major features such as:

```javascript
let
const
arrow functions
classes
template literals
destructuring
```

For your coding exam, you mainly need to recognize the history/version terminology and understand that modern JavaScript follows ECMAScript standards.

2. Variables and Data Types

🔥 EXTREMELY IMPORTANT

This is where your actual JavaScript programming begins.

Variables

A variable stores a value.

```javascript
let name = "Ajay";
let age = 20;
```

Think:

```
name → "Ajay"
age  → 20
```

`var`

Older JavaScript:

```javascript
var age = 20;
```

You can change it:

```javascript
var age = 20;
age = 21;
```

`let`

Modern JavaScript:

```javascript
let age = 20;
age = 21;
```

You can reassign it.

But you cannot redeclare the same `let` variable in the same scope:

```javascript
let age = 20;

// let age = 21; ❌
```

`const`

Used when the variable should not be reassigned:

```javascript
const pi = 3.14159;
```

This is invalid:

```javascript
const pi = 3.14159;
pi = 4;   // ❌
```

`var` vs `let` vs `const`

Keyword	Reassign?	Redeclare in same scope?
`var`	Yes	Yes
`let`	Yes	No
`const`	No	No

For modern coding, you will generally use:

```javascript
let
const
```

Data Types

JavaScript has several important data types.

Number

```javascript
let age = 20;
let price = 99.5;
```

Both are `number`.

String

```javascript
let name = "Ajay";
let college = 'SREC';
```

Strings can use:

```javascript
"Hello"
'Hello'
```

Boolean

```javascript
let passed = true;
let failed = false;
```

Only:

```
true
false
```

Undefined

```javascript
let x;
```

`x` currently has the value:

```
undefined
```

Null

```javascript
let x = null;
```

`null` represents an intentional absence of a value.

BigInt

For very large integers:

```javascript
let x = 12345678901234567890123456789n;
```

The `n` indicates a BigInt literal.

Symbol

Used for unique identifiers:

```javascript
let id = Symbol("id");
```

This is less likely to be a major coding question, but know that it is a JavaScript primitive type.

Object

```javascript
let student = {
    name: "Ajay",
    age: 20
};
```

Arrays and many other structured values are objects in JavaScript's type system.

`typeof`

Very important for coding questions.

```javascript
let age = 20;

console.log(typeof age);
```

Output:

```
number
```

Examples:

```javascript
typeof "Hello"      // "string"
typeof 10           // "number"
typeof true         // "boolean"
typeof undefined    // "undefined"
typeof {}           // "object"
```

One famous JavaScript behavior:

```javascript
typeof null
```

produces:

```
"object"
```

This is a historical language quirk.

3. Statements

A JavaScript program consists of statements.

Example:

```javascript
let age = 20;
console.log(age);
```

Each instruction performs some operation.

Conditional Statements

🔥 Very important.

`if`

```javascript
let age = 20;

if (age >= 18) {
    console.log("Eligible");
}
```

`if...else`

```javascript
let age = 16;

if (age >= 18) {
    console.log("Eligible");
} else {
    console.log("Not eligible");
}
```

`if...else if...else`

```javascript
let mark = 75;

if (mark >= 90) {
    console.log("A+");
} else if (mark >= 80) {
    console.log("A");
} else if (mark >= 70) {
    console.log("B");
} else {
    console.log("C");
}
```

`switch`

Useful when checking one expression against multiple fixed cases.

```javascript
let day = 2;

switch (day) {

    case 1:
        console.log("Monday");
        break;

    case 2:
        console.log("Tuesday");
        break;

    case 3:
        console.log("Wednesday");
        break;

    default:
        console.log("Invalid day");
}
```

Important:

```javascript
break;
```

prevents execution from continuing into the next case.

Loops

🔥🔥 Very important for coding.

`for`

```javascript
for (let i = 1; i <= 5; i++) {
    console.log(i);
}
```

Output:

```plain text
1
2
3
4
5
```

Understand the three sections:

```plain text
initialization
      ↓
condition
      ↓
update
```

```javascript
for (let i = 1; i <= 5; i++)
```

`while`

```javascript
let i = 1;

while (i <= 5) {
    console.log(i);
    i++;
}
```

`do...while`

```javascript
let i = 1;

do {
    console.log(i);
    i++;
} while (i <= 5);
```

The body executes at least once.

`break`

Stops a loop.

```javascript
for (let i = 1; i <= 10; i++) {

    if (i === 5) {
        break;
    }

    console.log(i);
}
```

Output:

```plain text
1
2
3
4
```

`continue`

Skips the current iteration.

```javascript
for (let i = 1; i <= 5; i++) {

    if (i === 3) {
        continue;
    }

    console.log(i);
}
```

Output:

```plain text
1
2
4
5
```

4. Operators

🔥🔥🔥 Very important for coding

You need to know the operator categories.

Arithmetic Operators

```javascript
let a = 10;
let b = 3;

console.log(a + b);
console.log(a - b);
console.log(a * b);
console.log(a / b);
console.log(a % b);
```

Operators:

```plain text
+   addition
-   subtraction
*   multiplication
/   division
%   remainder
```

Exponentiation

```javascript
console.log(2 ** 3);
```

Output:

```plain text
8
```

Increment

```javascript
let x = 5;

x++;
```

Now:

```plain text
x = 6
```

Decrement

```javascript
let x = 5;

x--;
```

Now:

```plain text
x = 4
```

Assignment Operators

```javascript
let x = 10;

x += 5;
```

Equivalent to:

```javascript
x = x + 5;
```

Similarly:

```javascript
x -= 5;
x *= 5;
x /= 5;
x %= 5;
```

Comparison Operators

```javascript
a > b
a < b
a >= b
a <= b
a == b
a != b
a === b
a !== b
```

`==` vs `===`

🔥 Very important.

```javascript
5 == "5"
```

is:

```
true
```

because `==` allows type conversion.

But:

```javascript
5 === "5"
```

is:

```
false
```

because `===` checks both value and type.

Prefer `===` when you want strict comparison.

Logical Operators

AND

```javascript
&&
```

Example:

```javascript
if (age >= 18 && age <= 60) {
    console.log("Valid age");
}
```

Both conditions must be true.

OR

```javascript
||
```

Example:

```javascript
if (day === "Saturday" || day === "Sunday") {
    console.log("Weekend");
}
```

At least one condition must be true.

NOT

```javascript
!
```

Example:

```javascript
let passed = false;

if (!passed) {
    console.log("Not passed");
}
```

Ternary Operator

Very useful.

```javascript
let age = 20;

let result = age >= 18 ? "Adult" : "Minor";
```

This is basically a compact:

```javascript
if...else
```

5. Literals

A literal is a value written directly in the source code.

For example:

```javascript
10
```

is a number literal.

```javascript
"Hello"
```

is a string literal.

```javascript
true
```

is a Boolean literal.

String Literals

```javascript
"Hello"
'Hello'
```

Template Literals

🔥 Important modern JavaScript feature.

Use backticks:

```javascript
let name = "Ajay";
let age = 20;

console.log(`My name is ${name} and I am ${age} years old.`);
```

Output:

```
My name is Ajay and I am 20 years old.
```

The `${}` syntax inserts expressions into the string.

Array Literals

```javascript
let marks = [90, 85, 95];
```

Object Literals

```javascript
let student = {
    name: "Ajay",
    age: 20
};
```

Regular Expression Literals

Later:

```javascript
let pattern = /hello/;
```

This is also a literal form.

6. Functions

🔥🔥🔥 One of the most important topics

A function is a reusable block of code.

Basic syntax:

```javascript
function functionName(parameters) {
    // code
}
```

Example:

```javascript
function greet() {
    console.log("Hello");
}

greet();
```

Function with Parameters

```javascript
function add(a, b) {
    console.log(a + b);
}

add(10, 20);
```

Output:

```
30
```

Here:

```
a, b   → parameters
10, 20 → arguments
```

Return Value

```javascript
function add(a, b) {
    return a + b;
}

let result = add(10, 20);

console.log(result);
```

Output:

```
30
```

Why `return` matters

Compare:

```javascript
function add(a, b) {
    console.log(a + b);
}
```

with:

```javascript
function add(a, b) {
    return a + b;
}
```

The second function gives the result back to the caller.

Function Expression

```javascript
const add = function(a, b) {
    return a + b;
};

console.log(add(10, 20));
```

Arrow Function

Modern JavaScript:

```javascript
const add = (a, b) => {
    return a + b;
};
```

Short form:

```javascript
const add = (a, b) => a + b;
```

Know the syntax because ES6/modern JavaScript questions may include it.

Default Parameters

```javascript
function greet(name = "Guest") {
    console.log("Hello " + name);
}

greet();
```

Output:

```
Hello Guest
```

7. Objects

🔥🔥 Important

Objects allow you to group related properties and behavior.

Example:

```javascript
let student = {
    name: "Ajay",
    age: 20,
    department: "CSE"
};
```

Accessing Properties

Dot notation:

```javascript
console.log(student.name);
```

Bracket notation:

```javascript
console.log(student["name"]);
```

Both access:

```
Ajay
```

Changing Properties

```javascript
student.age = 21;
```

Adding Properties

```javascript
student.mark = 95;
```

Now:

```javascript
let student = {
    name: "Ajay",
    age: 21,
    department: "CSE",
    mark: 95
};
```

Deleting Properties

```javascript
delete student.mark;
```

Methods

An object can contain functions.

```javascript
let student = {

    name: "Ajay",

    greet: function() {
        console.log("Hello");
    }

};

student.greet();
```

`this`

Inside an object method:

```javascript
let student = {

    name: "Ajay",

    greet: function() {
        console.log("Hello " + this.name);
    }

};

student.greet();
```

`this.name` refers to the `name` property of the relevant object in this method call.

This is important when studying objects.

8. Built-in Object — Array

🔥🔥🔥 Very important for coding

An array stores multiple values.

```javascript
let marks = [90, 80, 75, 95];
```

Indexes start at:

```
0
```

So:

```javascript
marks[0]
```

is:

```
90
```

and:

```javascript
marks[2]
```

is:

```
75
```

Array Length

```javascript
console.log(marks.length);
```

Changing an Element

```javascript
marks[1] = 85;
```

`push()`

Adds an element at the end.

```javascript
marks.push(100);
```

`pop()`

Removes the last element.

```javascript
marks.pop();
```

`unshift()`

Adds to beginning.

```javascript
marks.unshift(70);
```

`shift()`

Removes first element.

```javascript
marks.shift();
```

`indexOf()`

```javascript
let fruits = ["Apple", "Mango", "Orange"];

console.log(fruits.indexOf("Mango"));
```

Output:

```
1
```

`includes()`

```javascript
console.log(fruits.includes("Mango"));
```

Output:

```
true
```

`slice()`

Returns a portion without modifying the original array.

```javascript
let numbers = [10, 20, 30, 40, 50];

let result = numbers.slice(1, 4);

console.log(result);
```

Result:

```
[20, 30, 40]
```

`splice()`

Can remove/add elements in the original array.

```javascript
let numbers = [10, 20, 30, 40];

numbers.splice(1, 2);
```

Now:

```
[10, 40]
```

Looping Through Arrays

Traditional `for`:

```javascript
let marks = [90, 80, 70];

for (let i = 0; i < marks.length; i++) {
    console.log(marks[i]);
}
```

Very important coding pattern.

`for...of`

```javascript
for (let mark of marks) {
    console.log(mark);
}
```

This directly gives the values.

9. Built-in Object — Date

JavaScript provides the `Date` object.

```javascript
let today = new Date();

console.log(today);
```

Creating a Specific Date

```javascript
let date = new Date("2026-08-15");

console.log(date);
```

Date Components

```javascript
let date = new Date();

console.log(date.getFullYear());
console.log(date.getMonth());
console.log(date.getDate());
console.log(date.getDay());
```

Important:

```
getFullYear() → year
getMonth()    → month
getDate()     → day of month
getDay()      → day of week
```

Important month detail

`getMonth()` is zero-based:

```
January  = 0
February = 1
...
December = 11
```

Time Components

```javascript
console.log(date.getHours());
console.log(date.getMinutes());
console.log(date.getSeconds());
```

Setting Date Components

```javascript
date.setFullYear(2027);
date.setMonth(5);
date.setDate(20);
```

Know the major `get` and `set` methods.

10. Built-in Object — Math

🔥 Useful for coding questions.

`Math` provides mathematical functions and constants.

`Math.round()`

```javascript
console.log(Math.round(4.6));
```

Output:

```
5
```

`Math.floor()`

Rounds downward.

```javascript
console.log(Math.floor(4.9));
```

Output:

```
4
```

`Math.ceil()`

Rounds upward.

```javascript
console.log(Math.ceil(4.1));
```

Output:

```
5
```

`Math.trunc()`

Removes the fractional part.

```javascript
Math.trunc(4.9);
```

Result:

```
4
```

`Math.sqrt()`

```javascript
Math.sqrt(25);
```

Result:

```
5
```

`Math.pow()`

```javascript
Math.pow(2, 3);
```

Result:

```
8
```

You can also use:

```javascript
2 ** 3
```

`Math.abs()`

```javascript
Math.abs(-20);
```

Result:

```
20
```

`Math.max()`

```javascript
Math.max(10, 50, 20);
```

Result:

```
50
```

`Math.min()`

```javascript
Math.min(10, 50, 20);
```

Result:

```
10
```

`Math.random()`

Generates a pseudo-random number from:

```
0 inclusive
to
1 exclusive
```

Example:

```javascript
let number = Math.random();

console.log(number);
```

Random integer from 1 to 10:

```javascript
let number = Math.floor(Math.random() * 10) + 1;
```

This is a useful coding pattern.

11. Built-in Object — String

🔥🔥 Important

Create a string:

```javascript
let text = "JavaScript";
```

`length`

```javascript
console.log(text.length);
```

`toUpperCase()`

```javascript
console.log(text.toUpperCase());
```

`toLowerCase()`

```javascript
console.log(text.toLowerCase());
```

`charAt()`

```javascript
console.log(text.charAt(0));
```

Output:

```
J
```

`indexOf()`

```javascript
console.log(text.indexOf("Script"));
```

Returns the starting position.

`includes()`

```javascript
console.log(text.includes("Java"));
```

`startsWith()`

```javascript
text.startsWith("Java");
```

`endsWith()`

```javascript
text.endsWith("Script");
```

`slice()`

```javascript
let text = "JavaScript";

console.log(text.slice(0, 4));
```

Output:

```
Java
```

`substring()`

```javascript
console.log(text.substring(0, 4));
```

Output:

```
Java
```

`replace()`

```javascript
let text = "I like Java";

let result = text.replace("Java", "JavaScript");

console.log(result);
```

Output:

```
I like JavaScript
```

`trim()`

Removes whitespace from both ends.

```javascript
let text = "   Hello   ";

console.log(text.trim());
```

`split()`

🔥 Very useful in coding.

```javascript
let text = "HTML,CSS,JavaScript";

let result = text.split(",");

console.log(result);
```

Result:

```
["HTML", "CSS", "JavaScript"]
```

12. Regular Expressions

🔥 This is one of the more advanced topics in your portion.

A regular expression is a pattern used to search, match, or manipulate text.

Basic syntax:

```javascript
let pattern = /hello/;
```

`test()`

Checks whether the pattern exists.

```javascript
let pattern = /hello/;

console.log(pattern.test("hello world"));
```

Output:

```
true
```

Case-insensitive flag

```javascript
let pattern = /hello/i;

console.log(pattern.test("HELLO"));
```

`i` means case-insensitive matching.

Global flag

```javascript
let pattern = /hello/g;
```

`g` means global matching.

Character Classes

[abc]

Matches one of:

```
a
b
c
```

Example:

```javascript
let pattern = /[abc]/;
```

[0-9]

Any digit from 0 to 9.

```javascript
let pattern = /[0-9]/;
```

[a-z]

Lowercase letters.

```javascript
let pattern = /[a-z]/;
```

[A-Z]

Uppercase letters.

```javascript
let pattern = /[A-Z]/;
```

Quantifiers

+

One or more.

```javascript
/[0-9]+/
```

Matches:

```
123
4567
```

*

Zero or more.

```javascript
/[0-9]*/
```

?

Zero or one.

```javascript
/colou?r/
```

Can match:

```
color
colour
```

{n}

Exactly `n` times.

```javascript
/[0-9]{4}/
```

Matches four digits.

{n,m}

Between `n` and `m` occurrences.

```javascript
/[0-9]{2,4}/
```

Anchors

^

Beginning of string.

```javascript
/^Hello/
```

$

End of string.

```javascript
/world$/
```

Character shortcuts

```
\d → digit
\D → non-digit
\w → word character
\W → non-word character
\s → whitespace
\S → non-whitespace
```

Example:

```javascript
let pattern = /^\d+$/;
```

This matches a string containing only digits.

Regular Expression with String Methods

You can use:

```javascript
match()
search()
replace()
split()
```

Example:

```javascript
let text = "My number is 98765";

let result = text.match(/\d+/);

console.log(result);
```

13. Exceptions

🔥🔥 Final topic in your portion

Exception handling allows your program to respond to runtime errors in a controlled way.

The main structure is:

```javascript
try {

    // risky code

} catch (error) {

    // handle error

}
```

Basic Example

```javascript
try {

    let result = x + 10;

} catch (error) {

    console.log("An error occurred");

}
```

Because `x` has not been defined, the error is caught.

try...catch...finally

🔥 Very important.

```javascript
try {

    console.log("Trying");

} catch (error) {

    console.log("Error occurred");

} finally {

    console.log("Finished");

}
```

The `finally` block executes after the try/catch process.

throw

You can manually create an exception using:

```javascript
throw
```

Example:

```javascript
let age = 15;

if (age < 18) {
    throw new Error("Age must be 18 or above");
}
```

Complete Exception Example

```javascript
function checkAge(age) {

    try {

        if (age < 18) {
            throw new Error("Not eligible");
        }

        console.log("Eligible");

    } catch (error) {

        console.log(error.message);

    } finally {

        console.log("Checking completed");

    }
}

checkAge(16);
```

Output:

```
Not eligible
Checking completed
```

Error Object

JavaScript provides an `Error` object.

```javascript
throw new Error("Something went wrong");
```

Then:

```javascript
catch (error) {
    console.log(error.message);
}
```

`error.message` gives:

```
Something went wrong
```

Your JavaScript Portion as a Learning Tree

This is the structure I want you to keep in your head:

```
JAVASCRIPT
├── 1. History and Versions
│
├── 2. Variables and Data Types
│   ├── var
│   ├── let
│   ├── const
│   ├── Number
│   ├── String
│   ├── Boolean
│   ├── Undefined
│   ├── Null
│   ├── BigInt
│   ├── Symbol
│   ├── Object
│   └── typeof
│
├── 3. Statements
│   ├── if
│   ├── if-else
│   ├── else-if
│   ├── switch
│   ├── for
│   ├── while
│   ├── do-while
│   ├── break
│   └── continue
│
├── 4. Operators
│   ├── Arithmetic
│   ├── Assignment
│   ├── Comparison
│   ├── Logical
│   ├── Increment/Decrement
│   ├── Exponentiation
│   └── Ternary
│
├── 5. Literals
│   ├── Number
│   ├── String
│   ├── Boolean
│   ├── Array
│   ├── Object
│   ├── Template
│   └── RegExp
│
├── 6. Functions
│   ├── Declaration
│   ├── Parameters
│   ├── Arguments
│   ├── Return
│   ├── Function Expression
│   ├── Arrow Function
│   └── Default Parameters
│
├── 7. Objects
│   ├── Properties
│   ├── Access
│   ├── Modify
│   ├── Add
│   ├── Delete
│   ├── Methods
│   └── this
│
├── 8. Array
│   ├── Index
│   ├── length
│   ├── push
│   ├── pop
│   ├── shift
│   ├── unshift
│   ├── indexOf
│   ├── includes
│   ├── slice
│   ├── splice
│   └── iteration
│
├── 9. Date
│   ├── new Date()
│   ├── get methods
│   └── set methods
│
├── 10. Math
│   ├── round
│   ├── floor
│   ├── ceil
│   ├── trunc
│   ├── sqrt
│   ├── pow
│   ├── abs
│   ├── min
│   ├── max
│   └── random
│
├── 11. String
│   ├── length
│   ├── charAt
│   ├── indexOf
│   ├── includes
│   ├── startsWith
│   ├── endsWith
│   ├── slice
│   ├── substring
│   ├── replace
│   ├── trim
│   └── split
│
├── 12. Regular Expressions
│   ├── Pattern
│   ├── test
│   ├── Flags
│   ├── Character classes
│   ├── Quantifiers
│   ├── Anchors
│   └── Character shortcuts
│
└── 13. Exceptions
    ├── try
    ├── catch
    ├── finally
    ├── throw
    └── Error
```